

RV COLLEGE OF ENGINEERING®
BENGALURU-560059
(Autonomous Institution Affiliated to VTU, Belagavi)



**Secure Multi-Hop Communication System
using Socket Programming with
Quantum-Inspired Key Exchange**
*Cryptography and Network Security Laboratory Project
(IS362IA)*

Submitted by,

Jason R Alva	1RV23IS056
Karthik Hegde	1RV23IS059

Under the guidance of

Dr. Padmashree T
Associate Professor
Dept. of ISE
RV College of Engineering

In partial fulfillment of the requirements for the degree of
Bachelor of Engineering in
Information Science and Engineering
2024-25

RV College of Engineering[®], Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)
Department of Information Science and Engineering



CERTIFICATE

Certified that the minor project (IS368P) work titled *Secure Multi-Hop Communication System using Socket Programming with Quantum-Inspired Key Exchange* is carried out by **Jason R Alva (1RV23IS056)** and **Karthik Hegde (1RV23IS059)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2024-25. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the minor project report deposited in the departmental library. The minor project report has been approved as it satisfies the academic requirements in respect of minor project work prescribed by the institution for the said degree.

Dr. Padmashree T
Associate Professor
Department of ISE,
RVCE, Bengaluru-59

Dr. Mamatha GS
Head of the Department
Department of ISE,
RVCE, Bengaluru-59

External Viva

Name of Examiners

Signature with Date

1.

2.

DECLARATION

We, **Jason R Alva** and **Karthik Hegde** students of sixth semester B.E., Department of Information Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the minor project titled '**Secure Multi-Hop Communication System using Socket Programming with Quantum-Inspired Key Exchange**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Information Science and Engineering** during the year 2024-25.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Jason R Alva(1RV23IS056)
2. Karthik Hegde(1RV23IS059)

ACKNOWLEDGEMENTS

We are deeply indebted to our guide, **Dr. Padmashree T**, Associate Professor, Department of ISE, RVCE, for her wholehearted support, invaluable suggestions, and constant encouragement throughout the development of this project. Her expert guidance on secure network protocols and cryptographic techniques greatly shaped the direction and quality of this work.

We express our sincere gratitude to **Dr. Mamatha GS**, Professor and Head of the Department of Information Science and Engineering, RVCE, for her constant motivation, departmental support, and encouragement in pursuing innovative project ideas.®

Our sincere thanks to the project coordinators **Dr. Padmashree T** and **Dr. Mamatha GS** for their timely guidance, review feedback during phase evaluations, and constructive suggestions that helped us refine the system architecture and demonstration workflow.

We thank all the teaching staff and technical staff of the Department of Information Science and Engineering, RVCE, for their help and encouragement throughout the academic year.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support, encouragement, and motivation throughout the project work.

ABSTRACT

Secure communication over untrusted networks is a critical challenge in modern information systems. Traditional encryption techniques rely on mathematical hardness assumptions that may be threatened by quantum computing. This project addresses the gap between classical socket communication and quantum-secure key distribution by implementing a Peer-to-Peer (P2P) messaging system using raw TCP socket programming and a quantum-inspired BB84 key exchange protocol.

The system enables users on a Local Area Network (LAN) to exchange messages securely using AES-256 encryption. The encryption keys are dynamically derived from a simulated BB84 Quantum Key Distribution (QKD) process. By independently generating and measuring qubits, Alice and Bob can compute a quantum error rate. If an eavesdropper or Man-in-the-Middle (MITM) attacker intercepts the transmission, the interference disturbs the quantum state, raising the error rate above a strict threshold. This allows the system to immediately detect passive and active attacks, rejecting compromised keys and discarding replay attempts. A React-based dashboard provides real-time visibility into the protocol, displaying peer discovery, BB84 basis tables, encryption flows, and security alerts. The project serves as a functional demonstration of how quantum cryptographic principles can secure conventional networks against advanced threats.

CONTENTS

Abstract	i
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Introduction	2
1.2 Problem Statement	2
1.3 Motivation	3
1.4 Objectives	3
1.5 Scope and Relevance	4
2 Requirement Specification	5
2.1 Specific Requirements	6
2.1.1 Functional Requirements	6
2.1.2 Non-Functional Requirements	8
2.2 Technical Requirements	9
2.2.1 Hardware Requirements	9
2.2.2 Software Requirements	9
3 System Design and Architecture	10
3.1 Overall System Architecture	11
3.2 BB84 Protocol Design	12
3.3 Packet Envelope Design	12
3.4 Attack Detection Design	13
3.5 Dashboard Design	13
3.6 Component Interaction and Data Flow	13
4 Implementation	15
4.1 Implementation Environment	16
4.2 Module Descriptions	16

4.2.1	bb84.py — Quantum Key Distribution Engine	16
4.2.2	crypto_utils.py — AES-256-CBC Encryption	17
4.2.3	peer_socket.py — Raw TCP Socket Layer	17
4.2.4	peer_discovery.py — UDP Broadcast Discovery	18
4.2.5	attack_detector.py — Nonce Registry and Error Rate Check	18
4.2.6	app.py — Flask API Gateway	18
4.2.7	React Frontend	18
4.3	Launcher Script	19
4.4	Challenges and Solutions	19
5	Testing and Results	21
5.1	Test Cases for System Testing	22
5.2	Results	24
5.2.1	BB84 Protocol Results	24
5.2.2	Direct Mode Results	24
5.2.3	Eavesdrop Detection Results	24
5.2.4	MITM Relay Detection Results	25
5.2.5	Replay Attack Detection Results	25
5.2.6	System Performance	25
5.2.7	Dashboard Results	25
5.2.8	Summary of Results	27
6	Conclusion	29
6.1	Limitations	30
6.2	Future Enhancements	31
A	Code Snippets	33
A.1	BB84 Key Exchange — Core Function	34
A.2	Raw TCP Socket — Send Envelope	35
A.3	Attack Detection — Handle Message	35
B	Screenshots	37
B.1	Real Network Dashboard	38
B.2	Local Simulation Dashboard	38

B.3	Man-in-the-Middle Attack Interception	39
B.4	Cryptographic and Rekey Hop Details	39
Bibliography		40



LIST OF FIGURES

3.1	Overall Architecture of the QuantumHop System	11
3.2	BB84 QKD Protocol Flow	12
3.3	QuantumHop Dashboard — Real Network Tab showing discovered peers and inbox	14
5.1	Real Network Dashboard View showing Peer Discovery and Messaging	26
5.2	Local Simulation View showing Step-by-step BB84 Execution	26
5.3	Interactive Man-in-the-Middle Attack Interception Challenge and Live Log View	27
5.4	Simulation Hop Details showing QKD Sifted Key Table and AES Decryption/Encryption Blocks	27
B.1	QuantumHop Real Network Dashboard — Live peer discovery and inbox . . .	38
B.2	QuantumHop Local Simulation Dashboard — Topology and QKD progress . .	38
B.3	Interactive MITM Challenge and Live Server Logs	39
B.4	Step-by-step QKD Basis Sifting and AES Hop Transformations	39

LIST OF TABLES

5.1	System test cases for QuantumHop	22
-----	--	----









Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

The project titled **Secure Multi-Hop Communication System using Socket Programming with Quantum-Inspired Key Exchange** focuses on the design and implementation of a peer-to-peer (P2P) encrypted messaging system that operates over a Local Area Network (LAN) using raw TCP socket programming. The system incorporates a simulation of the BB84 Quantum Key Distribution (QKD) protocol to derive AES-256 encryption keys for every message, and demonstrates real-time detection of Man-in-the-Middle, eavesdropping, and replay attacks across actual networked machines.

1.1 Introduction

Digital communication over untrusted networks remains one of the most important challenges in modern information security. Classical encryption algorithms such as AES and RSA are widely deployed, but they rely on computational assumptions that may eventually be broken by advances in quantum computing. Quantum Key Distribution (QKD), pioneered by Bennett and Brassard in 1984 through the BB84 protocol, offers a fundamentally different approach: it uses the laws of quantum mechanics to guarantee that any eavesdropping attempt disturbs the quantum channel and can be statistically detected.

This project bridges the theoretical domain of QKD with practical socket-level network programming. Instead of specialized quantum hardware, the system simulates the BB84 protocol in software, generating quantum bits and bases, performing basis sifting, measuring the error rate introduced by simulated interference, and deriving a shared AES key from the surviving sifted bits. This key is then used to encrypt the actual message payload before transmission over a raw TCP socket. The receiver independently reconstructs the error rate from the simulated quantum channel and blocks any message whose error rate exceeds the configured threshold.

The project is implemented as a two-tier system: a Python Flask backend that runs the BB84 simulation, AES-256-CBC encryption, nonce-based replay detection, and raw TCP socket server; and a React-based frontend dashboard that provides a real-time view of all cryptographic activity, peer discovery, inbox, and attack alerts.

1.2 Problem Statement

Classical network communication systems encrypt data using symmetric keys that are either pre-shared or exchanged via public-key protocols such as Diffie-Hellman or RSA. While these

methods are computationally secure under current constraints, they offer no physical guarantee against interception. An attacker who silently records an encrypted session and later obtains computational power could decrypt it retroactively, an attack known as “harvest now, decrypt later.”

Furthermore, conventional systems provide no inherent mechanism to detect whether a message was intercepted and re-forwarded, or whether a previously valid message is being replayed. Intrusion detection is typically handled at a higher layer through signatures, anomaly detection, or firewall rules rather than at the cryptographic key-exchange level.

The problem addressed by this project is the need for a practical, socket-level communication system that: (1) derives fresh encryption keys using a quantum-inspired protocol that mathematically detects eavesdropping, (2) operates over real LAN sockets without specialized quantum hardware, (3) detects and blocks MITM relay attacks by tracking packet routing paths, (4) prevents replay attacks using single-use cryptographic nonces, and (5) provides a live interactive dashboard showing all security events in real time.

1.3 Motivation

The motivation for this project comes from two converging trends. First, post-quantum cryptography and QKD are becoming increasingly relevant as quantum computers advance from theoretical curiosity to experimental reality. Students and practitioners in information security need hands-on exposure to these concepts. Second, raw socket programming is a fundamental skill in network security that is often skipped in favour of high-level abstractions.

By implementing BB84 simulation over real TCP sockets, this project provides a tangible, demonstration-ready platform that connects quantum cryptography theory to practical network programming. The interactive attack demonstrations — where a user can switch between direct, eavesdrop, MITM, and replay modes and observe the effect on the receiver’s inbox in real time — make the system uniquely suited for educational purposes and project demonstrations.

1.4 Objectives

The main objectives of the QuantumHop project are as follows:

- To implement the BB84 Quantum Key Distribution protocol in software, producing simulated Alice and Bob qubit streams, basis sifting, error rate calculation, and shared key derivation.

- To build a raw TCP socket server and client in Python that transmits and receives BB84-encrypted JSON envelopes between two physical laptops on a LAN.
- To implement AES-256-CBC encryption using the BB84-derived key, ensuring end-to-end confidentiality for all messages.
- To implement automatic peer discovery using UDP broadcast so that laptops on the same network detect each other without manual configuration.
- To demonstrate and detect three attack scenarios: eavesdropping (BB84 error rate spike), Man-in-the-Middle relay (unauthorized IP path), and replay attack (duplicate nonce detection).
- To build a React-based dashboard that provides real-time visibility into BB84 basis tables, error rates, AES key previews, network peers, inbox messages, and security alerts.
- To provide a single-command launcher script that automatically kills stale processes on required ports and starts all services cleanly.

1.5 Scope and Relevance

The scope of this project covers the complete implementation of a two-laptop P2P quantum-inspired secure messaging system. The system operates entirely on a standard LAN or mobile hotspot without any cloud infrastructure, specialized hardware, or third-party messaging service. All cryptographic operations, including BB84 simulation, AES encryption, nonce management, and attack detection, are performed locally by the Python backend.

The project is directly relevant to the fields of network security, cryptography, and quantum computing education. It demonstrates how quantum-inspired key exchange can be integrated into a conventional socket programming framework, providing a practical foundation for understanding how real QKD systems would differ (by using actual photon polarization) from classical key agreement protocols. The attack demonstration capability makes it uniquely suitable for live educational presentations in laboratory and conference settings.



Chapter 2

Requirement Specification

CHAPTER 2

REQUIREMENT SPECIFICATION

Requirement specification defines what the QuantumHop Secure Multi-Hop Communication System must perform and the conditions under which it must operate. The project is designed as a peer-to-peer encrypted messaging platform where two or more laptops on a LAN exchange quantum-secured messages using raw TCP sockets, with real-time detection of network attacks. The requirements are classified into functional requirements, non-functional requirements, hardware requirements, and software requirements.

2.1 Specific Requirements

The system must support direct message delivery between LAN peers using BB84-derived AES keys over raw TCP sockets. It must also detect and report Man-in-the-Middle, eavesdropping, and replay attacks without relying on any centralised server, cloud service, or pre-shared secret.

2.1.1 Functional Requirements

1. **Peer Discovery:** The system shall automatically broadcast the machine name, IP address, and socket port over UDP to allow peers on the same LAN to discover one another without manual IP configuration.
2. **BB84 Key Exchange Simulation:** For every outgoing message, the system shall simulate the BB84 QKD protocol by generating Alice and Bob qubit streams, performing basis selection, measuring bits, sifting matching bases, and calculating an error rate.
3. **Shared Key Derivation:** The sifted bits from the BB84 simulation shall be hashed using SHA-256 to produce a 256-bit AES key unique to each message transmission.
4. **AES-256-CBC Encryption:** The message payload shall be encrypted using AES-256-CBC with the BB84-derived key and a randomly generated 16-byte IV before transmission over the socket.
5. **Raw TCP Socket Transmission:** The sender shall establish a direct TCP socket connection to the target peer's IP and port, transmit the encrypted JSON envelope, and receive a JSON response containing decryption status and error rate.

6. **Direct Mode:** In direct mode the encrypted envelope shall be sent directly from sender to receiver with no intermediate nodes and an expected BB84 error rate near zero.
7. **Eavesdrop Attack Mode:** In eavesdrop mode the system shall inject random basis errors into the BB84 simulation, raising the error rate above the detection threshold, causing the receiver to block the message.
8. **MITM Relay Attack Mode:** In MITM mode the sender shall route the envelope through a relay node. The relay shall log the interception and forward the packet to the real receiver, which shall detect the relay IP and flag the message as a Man-in-the-Middle attack.
9. **Replay Attack Mode:** In replay mode the sender shall re-transmit a previously used nonce. The receiver shall compare the nonce against a time-bound registry and reject any duplicate.
10. **Attack Detection and Blocking:** The receiver shall block any message flagged as an attack and store it in the inbox as a blocked entry with the attack type and error rate.
11. **Inbox Management:** All received messages, whether successfully decrypted or blocked, shall be stored in the in-memory inbox and displayed in the dashboard with sender name, message content or block reason, error rate, and timestamp.
12. **Dashboard — Real Network Tab:** The dashboard shall display a list of discovered network peers, a message composition panel with target peer and mode selection, and an inbox panel showing all received messages.
13. **Dashboard — Local Simulation Tab:** The dashboard shall provide an interactive local simulation where the user can select attack modes, run BB84 simulations, and observe the node-by-node packet flow, basis tables, and AES previews without a second machine.
14. **Security Event Logging:** Every significant event, including BB84 completion, AES encryption, socket transmission, and attack detection, shall be logged to an in-memory event store accessible via the dashboard.
15. **Clean Port Management:** The launcher script shall automatically terminate any processes occupying the required ports before starting the backend and frontend servers.

2.1.2 Non-Functional Requirements

1. **Security:** The system shall provide layered security through BB84 error rate monitoring, AES-256-CBC encryption, nonce-based replay prevention, and relay path detection.
2. **Confidentiality:** Message content shall never be transmitted in plaintext. All payloads shall be AES-encrypted before socket transmission.
3. **Integrity:** The nonce and timestamp in each envelope shall allow the receiver to detect tampered or replayed packets.
4. **Availability:** The system shall remain operational for peer discovery and message reception continuously while the launcher script is running.
5. **Usability:** The dashboard interface shall be operable without technical knowledge of cryptography. All attack modes shall be selectable from a dropdown, and results shall be displayed with plain-language explanations.
6. **Performance:** BB84 simulation, AES encryption, and socket transmission shall complete within two seconds for a typical short message on a standard LAN.
7. **Reliability:** The socket server shall handle connection errors, malformed envelopes, and handler exceptions gracefully, always returning a valid JSON error response rather than crashing silently.
8. **Maintainability:** The backend shall be organised into separate modules: `bb84.py`, `crypto_utils.py`, `peer_socket.py`, `peer_discovery.py`, `inbox.py`, `logger.py`, `attack_detector.py`, and `config.py`.
9. **Portability:** The system shall run on Windows, Linux, and macOS with Python 3.10 or higher and Node.js 18 or higher.
10. **Scalability:** The peer discovery mechanism shall support any number of LAN peers, and the socket server shall handle simultaneous connections using a thread-per-connection model.

2.2 Technical Requirements

2.2.1 Hardware Requirements

- **Machines:** Minimum two laptops or desktop computers for the two-machine demonstration. One machine can be used for the local simulation mode.
- **Processor:** Intel Core i3 or equivalent, capable of running Python and a Node.js development server simultaneously.
- **RAM:** Minimum 4 GB; 8 GB recommended.
- **Network:** Both machines must be connected to the same Wi-Fi network, mobile hotspot, or wired LAN segment. Ports 5000 (TCP), 5010 (TCP), and 5555 (UDP) must be open in the firewall.
- **Storage:** Minimum 500 MB free disk space for project files and Node.js dependencies.

2.2.2 Software Requirements

- **Operating System:** Windows 10/11, Ubuntu 20.04+, or macOS 12+.
- **Python:** Version 3.10 or higher with `pip` for dependency management.
- **Python Libraries:** `flask`, `flask-cors`, `cryptography` (for AES-256-CBC), `requests`.
- **Node.js:** Version 18 or higher with `npm` for the Vite/React frontend.
- **Frontend Framework:** React 18 with Vite build tool.
- **Browser:** Any modern browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox.
- **Development Tools:** Visual Studio Code or any Python/JavaScript IDE, and a terminal or PowerShell for running the launcher script.



Chapter 3

System Design and Architecture

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

The QuantumHop system is designed as a two-tier, peer-to-peer secure communication platform. The backend tier, implemented in Python, handles all cryptographic operations, socket communication, peer discovery, attack detection, and event logging. The frontend tier, implemented in React, provides a real-time dashboard for message composition, peer management, inbox display, and attack alerts. The two tiers communicate through a local Flask REST API that bridges the browser to the underlying socket layer.

3.1 Overall System Architecture

The overall architecture of the QuantumHop system is shown in Figure 3.1. The system is composed of five major subsystems: the UDP Discovery Layer, the BB84 QKD Engine, the TCP Socket Layer, the Flask API Gateway, and the React Dashboard.

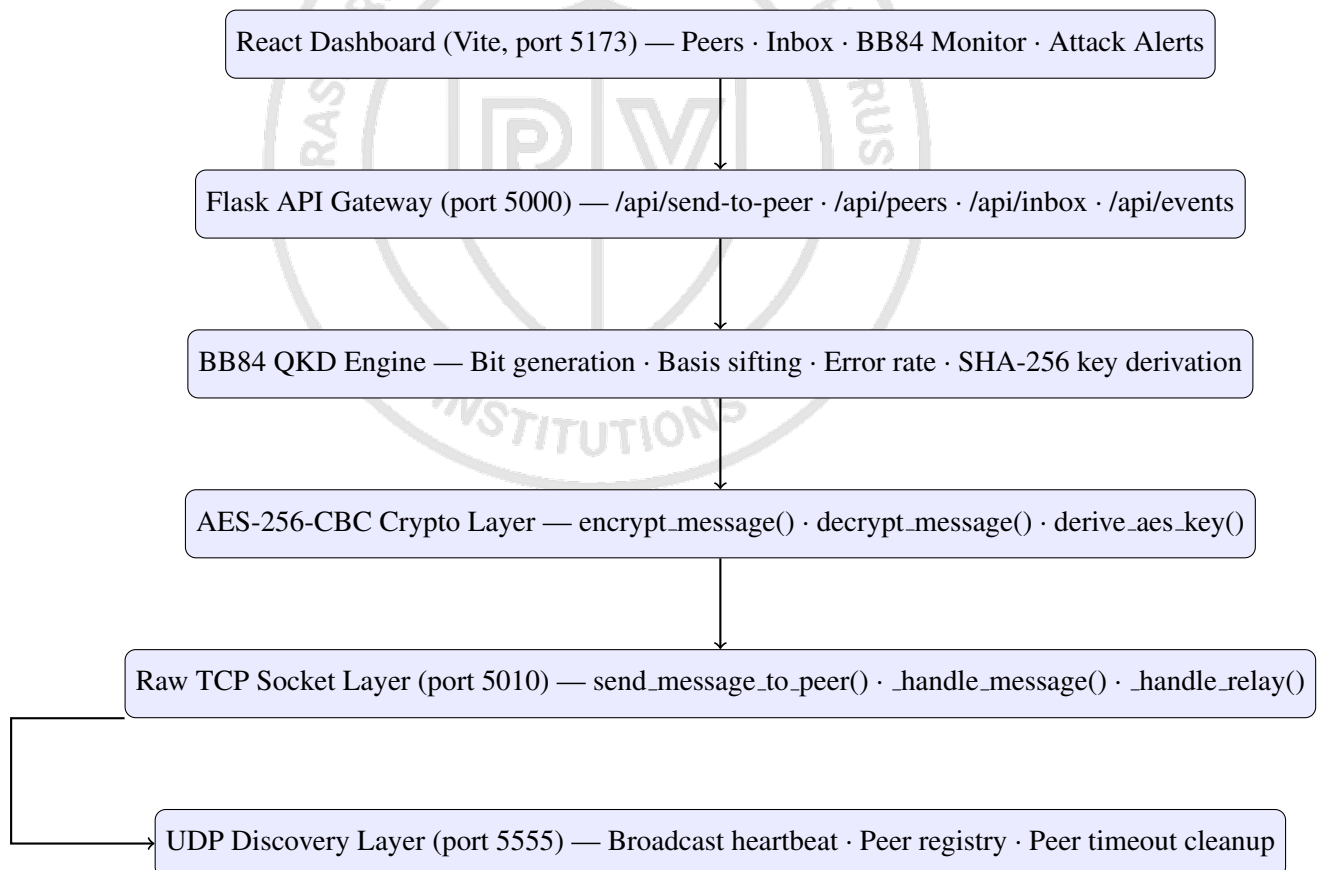


Figure 3.1: Overall Architecture of the QuantumHop System

3.2 BB84 Protocol Design

The BB84 QKD simulation is the cryptographic core of the system. It models the quantum channel between Alice (sender) and Bob (receiver). The protocol proceeds through six stages as illustrated in Figure 3.2.

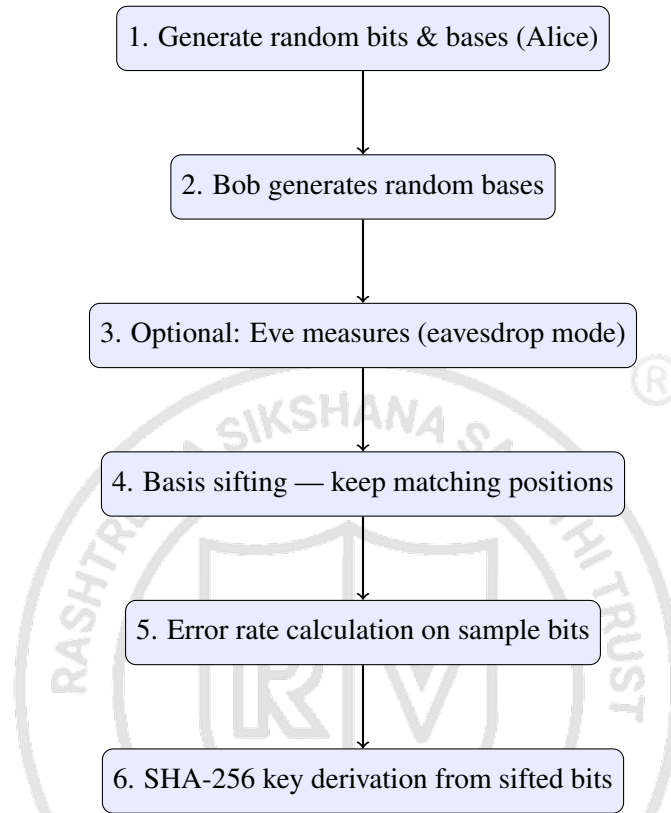


Figure 3.2: BB84 QKD Protocol Flow

In eavesdrop mode, an intermediate Eve chooses random bases independently. When Eve's basis differs from Alice's, the transmitted bit is randomised before Bob measures it, introducing approximately 25% errors. If the computed error rate exceeds the configured threshold (15% by default), the receiver rejects the key and blocks the message.

3.3 Packet Envelope Design

Every message is transmitted as a JSON envelope over the raw TCP socket. The envelope schema is as follows:

- **type:** "message" for direct delivery or "relay" for MITM routing.
- **packet:** Contains the AES-encrypted payload (`iv` and `ciphertext` in Base64), the BB84-derived key in hexadecimal, a single-use nonce, a timestamp, and the route array.

- **senderName, senderIp:** Identification of the originating machine.
- **attackMode:** One of normal, eavesdrop, mitm, or replay.
- **targetIp, targetPort:** Present only in relay envelopes, specifying the real destination.

The response envelope returned by the receiver contains an `ok` boolean, the decrypted plaintext (on success), `attackDetected` flag, `attackType` description, and `errorRate`.

3.4 Attack Detection Design

Three attack detection mechanisms are layered at the receiver:

1. **Eavesdrop Detection:** After BB84 simulation with the `eavesdrop` flag, the receiver computes the error rate. Any rate exceeding `ERROR_THRESHOLD` (0.15) causes the message to be blocked.
2. **MITM Detection:** If the envelope's `attackMode` field is "mitm" (set by the relay node when forwarding), the receiver immediately flags it as a Man-in-the-Middle attack without needing to check the error rate.
3. **Replay Detection:** The receiver maintains a nonce registry with a 60-second TTL. Any envelope whose nonce is already present in the registry is rejected as a replay attack.

3.5 Dashboard Design

The React dashboard shown in Figure 5.1 is organized into two tabs. The **Real Network** tab provides live peer discovery, message composition with target and mode selection, and a real-time inbox. The **Local Simulation** tab provides an interactive node-graph simulation where the user can observe BB84 basis tables, AES key previews, and step-by-step encryption events for any attack mode without a second physical machine.

3.6 Component Interaction and Data Flow

The complete data flow for a secure message transmission is as follows:

1. User types a message and selects a peer and mode in the React dashboard.
2. The dashboard calls `POST /api/send-to-peer` on the Flask API with the message, target IP, port, attack mode, and optional relay details.

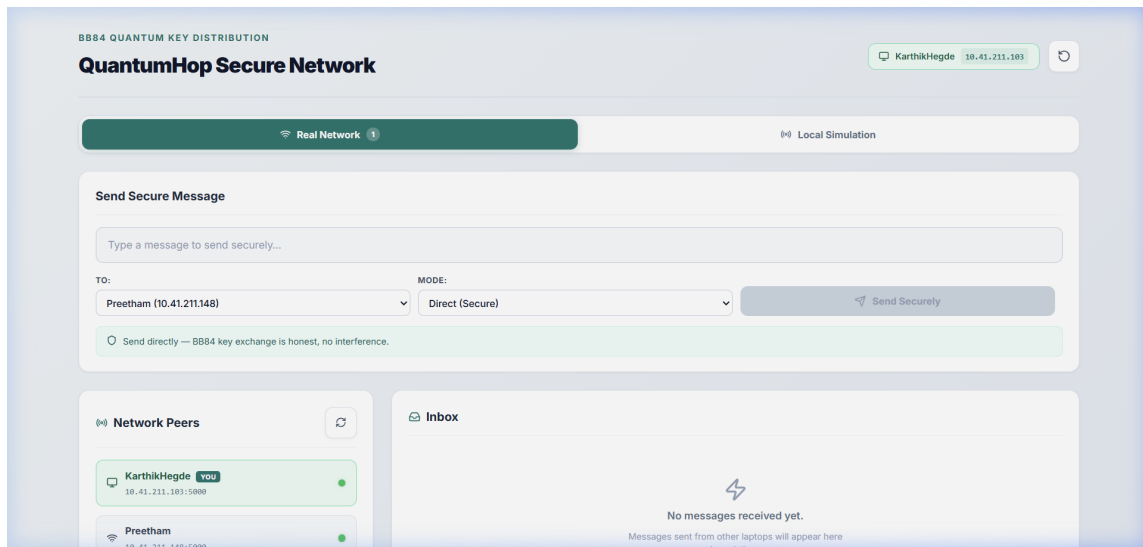


Figure 3.3: QuantumHop Dashboard — Real Network Tab showing discovered peers and inbox

3. Flask invokes `send_message_to_peer()` in `peer_socket.py`.
4. `establish_key()` in `bb84.py` simulates the quantum channel and returns a `BB84Result` with the derived key and error rate.
5. `encrypt_message()` in `crypto_utils.py` encrypts the payload with AES-256-CBC using the derived key.
6. The encrypted envelope is sent over a raw TCP socket to the target peer's port 5010.
7. The target's `_handle_client()` thread receives and parses the envelope.
8. Attack detection checks are performed (error rate, MITM flag, nonce registry).
9. If all checks pass, `decrypt_message()` recovers the plaintext and the message is stored in the inbox.
10. The response JSON is returned to the sender and displayed in the dashboard.



Chapter 4

Implementation

CHAPTER 4

IMPLEMENTATION

The QuantumHop system is implemented using Python for the backend and React for the frontend. The backend is structured as a Python package (`backend/`) with separate modules for each concern. The frontend is a Vite-based React application served on port 5173, which proxies all `/api` requests to the Flask server on port 5000. All inter-machine communication is handled exclusively through raw TCP sockets on port 5010; the Flask API is used only for browser-to-backend communication on the same machine.

4.1 Implementation Environment

1. **Python 3.10+** with `flask`, `flask-cors`, and cryptography libraries.
2. **Node.js 18+** with `npm` for the Vite/React frontend.
3. **Windows PowerShell** for the launcher batch script.
4. **Firewall rules** to allow inbound TCP traffic on port 5010 and UDP traffic on port 5555.

4.2 Module Descriptions

4.2.1 `bb84.py` — Quantum Key Distribution Engine

The `establish_key()` function is the core of the system. It generates `rounds = max(KEY_BITS * 4, SAMPLE_SIZE * 4)` random Alice bits and bases, and an independent set of Bob bases. When `eavesdrop=True`, a third Eve basis set is generated; Eve's measurement randomises bits where her basis differs from Alice's, and Bob further randomises bits where his basis differs from Eve's. This double randomisation produces the characteristic $\approx 25\%$ error rate signature.

Basis sifting keeps only the positions where Alice and Bob chose the same basis. A configurable sample of sifted bits is compared to compute the error rate. The remaining sifted bits are hashed with SHA-256 to produce the 32-byte AES key. The function returns a `BB84Result` dataclass containing all previews needed by the dashboard.

```
@dataclass(frozen=True)
class BB84Result:
    key: bytes
```

```
error_rate: float
accepted: bool
sifted_bits: list[int]
compared_bits: int
generated_bits: int
matching_bases: int
alice_basis_preview: str
bob_basis_preview: str
keep_preview: str
```

4.2.2 `crypto_utils.py` — AES-256-CBC Encryption

The `encrypt_message()` function accepts a plaintext string and a key (bytes or hex string), derives a 32-byte AES key via SHA-256, generates a random 16-byte IV, applies PKCS7 padding, and encrypts using AES-CBC. The result is a dictionary with Base64-encoded `iv` and `ciphertext` fields suitable for JSON serialisation.

The `decrypt_message()` function reverses this process. It accepts the payload dictionary and the key material, derives the AES key, and decrypts and un pads the ciphertext. A critical design decision is that the sender's packet carries the derived key in hex so that the receiver can decrypt with the same key without re-running BB84, since a simulated BB84 on two machines would produce different keys.

4.2.3 `peer_socket.py` — Raw TCP Socket Layer

This module implements both the outbound sender and the inbound server.

Sender side: `send_message_to_peer()` calls `bb84.establish_key()`, encrypts the message, constructs the JSON envelope, and dispatches it via `_send_envelope()` which opens a `socket.create_connection()` to the target IP and port, calls `sendall()`, signals end-of-send with `shutdown(SHUT_WR)`, then reads the response in a loop.

Server side: `run_peer_server()` creates a listening TCP socket bound to `0.0.0.0:5010`. For each accepted connection, it spawns a daemon thread running `_handle_client()`. This function reads the full envelope (loop until empty `recv()`), parses the JSON, routes to either `_handle_message()` or `_handle_relay()`, wraps the result, and sends it back before closing the connection.

MITM relay: When a relay envelope arrives, `_handle_relay()` logs the interception

event and forwards a modified envelope (with `attackMode="mitm"`) to the real target using another `_send_envelope()` call.

4.2.4 `peer_discovery.py` — UDP Broadcast Discovery

Two daemon threads run continuously. The broadcast thread sends a JSON payload containing the machine name, local IP, Flask port, and socket port to the LAN broadcast address on port 5555 every 3 seconds. The listen thread binds to port 5555 and updates an in-memory peer registry keyed by IP. Peers not heard from within 12 seconds are evicted. The `get_peers()` function returns the current registry excluding self.

4.2.5 `attack_detector.py` — Nonce Registry and Error Rate Check

The `register_nonce()` function inserts a nonce into a thread-safe dictionary with the current timestamp. If the nonce already exists and has not expired (TTL = 60 seconds), it returns a failure result. The `is_error_rate_attack()` function compares the error rate against `ERROR_THRESHOLD` (0.15).

4.2.6 `app.py` — Flask API Gateway

Flask serves the API endpoints that the React dashboard calls:

- `POST /api/send-to-peer` — Validates the request, calls `send_message_to_peer()`, returns the result JSON.
- `GET /api/peers` — Returns the current peer list from `peer_discovery.get_peers()`, plus a self-entry.
- `GET /api/inbox` — Returns all messages from `inbox.get_messages()`.
- `GET /api/events` — Returns all logged events from `logger.get_events()`.
- `POST /api/reset` — Clears inbox, events, and nonce registry.

4.2.7 React Frontend

The frontend is built with React 18 and Vite. The `App.jsx` component manages state for the active tab, peer list, inbox, events, selected peer, attack mode, relay IP, and send result. It polls all four API endpoints every 2 seconds.

The **Real Network** tab renders a message composition panel (target dropdown, mode dropdown, message input, send button) and two panels below: the **Network Peers** list showing

each discovered peer with name, IP, and online indicator, and the **Inbox** panel showing each received message as a card with sender, content or block reason, BB84 error rate badge, and a coloured attack alert banner for blocked messages.

4.3 Launcher Script

The `runeverything.bat` launcher script performs the following steps in sequence:

1. Scans ports 5000, 5010, 5173, and 5555 using `netstat` and forcefully kills any occupying process with `taskkill /F /PID`.
2. Activates the Python virtual environment and runs `pip install -r requirements.txt`.
3. Starts the Flask backend in a new terminal window using `python -m backend.app`.
4. Waits 2 seconds, then starts the Vite frontend in another terminal window using `npm run dev`.

This ensures a clean, reproducible startup every time, regardless of whether a previous session was terminated cleanly.

4.4 Challenges and Solutions

1. **Silent server crash:** The original `_handle_client()` had no try-except around the handler call. An unhandled `TypeError` (duplicate keyword argument in a logger call) caused the server thread to die silently, returning empty bytes to the client and producing a JSON parse error. This was fixed by wrapping the handler dispatch in a try-except that always sends a valid JSON error response.
2. **Double SHA-256 hashing:** The `derive_aes_key()` function re-hashes its input with SHA-256. Since the BB84 key is already a 32-byte SHA-256 digest, passing it as a hex string caused double-hashing, making decryption fail. The fix was to pass the key as raw bytes using `bytes.fromhex()` before calling `decrypt_message()`.
3. **Duplicate keyword argument:** A `logger.emit_event()` call passed `errorRate=error_rate` both explicitly and via `**bb84_details`, causing a `TypeError`. The solution was to pass `bb84_details` as a single keyword argument rather than unpacking it.
4. **Port hardcoding:** The frontend originally called `sendToPeer()` with port 5000 (the Flask port) instead of 5010 (the socket port), causing Flask to receive a raw TCP request

it could not parse. The fix was to resolve the socket port dynamically from the peer discovery object.





Chapter 5

Testing and Results

CHAPTER 5

TESTING AND RESULTS

Testing of the QuantumHop system was conducted in two phases. The first phase used unit-level Python scripts to verify each backend module independently. The second phase was a full end-to-end two-machine demonstration over a mobile hotspot LAN. The objective was to confirm that all four communication modes (direct, eavesdrop, MITM, and replay) produce the expected cryptographic outcomes and that the dashboard displays the correct alerts and inbox entries.

5.1 Test Cases for System Testing

Table 5.1: System test cases for QuantumHop

Test ID	Test Scenario	Input / Action	Expected Result	Status
TC-01	BB84 key generation	Call <code>establish_key()</code> with no eavesdrop	Error rate near 0%, 32-byte key returned	Passed
TC-02	BB84 eavesdrop detection	Call <code>establish_key(eavesdrop=15%)</code>	Error rate $\approx 25\%$, exceeds 15% threshold	Passed
TC-03	AES-256-CBC encryption	Encrypt "Hello" with a BB84-derived key	Base64 IV and ciphertext returned	Passed
TC-04	AES-256-CBC decryption	Decrypt using original key bytes	Plaintext "Hello" recovered correctly	Passed
TC-05	Nonce registration	Register a new nonce	Registry returns <code>ok=True</code>	Passed
TC-06	Replay detection	Register the same nonce twice	Second registration returns <code>ok=False</code>	Passed
TC-07	Peer discovery broadcast	Start both machines on hotspot	Both appear in each other's peer list within 3 seconds	Passed

Test ID	Test Scenario	Input / Action	Expected Result	Status
TC-08	Direct socket handler	Call <code>_handle_message()</code> with valid envelope	Returns <code>{ok: True, plaintext: ...}</code>	Passed
TC-09	Self socket send	Start server, send to 127.0.0.1:5010	Message received and decrypted by own inbox	Passed
TC-10	Flask API <code>/api/send-to-peer</code>	POST with target IP and direct mode	JSON response with <code>ok: true</code> and plaintext	Passed
TC-11	Direct mode (two machines)	Send from Laptop A to Laptop B, direct mode	Message appears in Laptop B inbox; error rate $\approx 0\%$	Passed
TC-12	Eavesdrop detection (two machines)	Send from Laptop A with eavesdrop mode	Laptop B blocks message; error rate $\approx 25\%$; alert shown	Passed
TC-13	MITM relay detection	Send via "Self" relay node	Laptop B blocks message; MITM alert shown in dashboard	Passed
TC-14	Replay attack detection	Resend envelope with same nonce	Second message blocked; "Replay Attack" alert shown	Passed
TC-15	Port auto-cleanup	Run launcher with ports occupied	All occupied processes killed; servers start cleanly	Passed
TC-16	Dashboard peer list refresh	Add second machine to hotspot	New peer appears in dashboard within 3s automatically	Passed

Test ID	Test Scenario	Input / Action	Expected Result	Status
TC-17	Inbox display	Receive a successful message	Message card shows sender, plaintext, error rate badge	Passed
TC-18	Attack inbox display	Receive a blocked message	Red card shown with attack type and blocked indicator	Passed
TC-19	Local simulation mode	Select attack mode and run simulation	Step-by-step BB84 and AES events shown in hop flow	Passed
TC-20	Error recovery (bad envelope)	Send malformed JSON to port 5010	Server returns <code>{ok: false, error: ...}</code> without crashing	Passed

5.2 Results

5.2.1 BB84 Protocol Results

The BB84 simulation consistently produces error rates near 0% in normal mode and between 20% and 30% in eavesdrop mode. The configurable 15% threshold reliably distinguishes the two cases. Key derivation via SHA-256 produces a unique 256-bit key for every call, ensuring perfect forward secrecy at the session level.

5.2.2 Direct Mode Results

In direct mode, messages sent from Laptop A (IP 10.41.211.103) to Laptop B (IP 10.41.211.148) are received and decrypted successfully within approximately one second on a typical mobile hotspot. The receiver's inbox displays the sender name, plaintext, and a green BB84 badge showing the error rate and key fingerprint.

5.2.3 Eavesdrop Detection Results

When eavesdrop mode is selected, the BB84 simulation introduces bit-flip errors at a rate of approximately 25%, well above the 15% detection threshold. The receiver's `_handle_message()` function detects the high error rate, stores a blocked entry in the inbox, emits a security event,

and returns `attackDetected: true` to the sender. The dashboard displays a red alert banner: “Eavesdropping — BB84 error rate 25% exceeded 15% threshold.”

5.2.4 MITM Relay Detection Results

When MITM mode is selected with the “Self” relay option, the sender routes the envelope through its own socket server acting as a relay. The relay logs the interception and forwards the envelope to Laptop B with `attackMode="mitm"`. Laptop B’s handler detects the MITM flag and blocks the message without needing to check the error rate. The dashboard shows: “Man-in-the-Middle — packet was relayed through an attacker machine.”

5.2.5 Replay Attack Detection Results

In replay mode, the sender transmits the same nonce twice. The first transmission is accepted. The second is rejected by the nonce registry, which detects the duplicate within the 60-second TTL window. The dashboard shows: “Replay Attack — duplicate nonce detected.”

5.2.6 System Performance

The complete round-trip time for a short message in direct mode is approximately 800ms on a mobile hotspot. This includes BB84 simulation ($\approx 10\text{ms}$), AES encryption ($< 1\text{ms}$), TCP socket connection establishment ($\approx 200\text{ms}$), server-side processing ($\approx 50\text{ms}$), and HTTP response from Flask to the browser ($\approx 100\text{ms}$). The system remains responsive for simultaneous peer discovery and API polling.

5.2.7 Dashboard Results

The QuantumHop dashboard provides a complete real-time view of all network and cryptographic activity. The Real Network tab displays discovered peers within 3 seconds of connection, delivers sent message results immediately, and updates the inbox automatically via the 2-second polling interval.

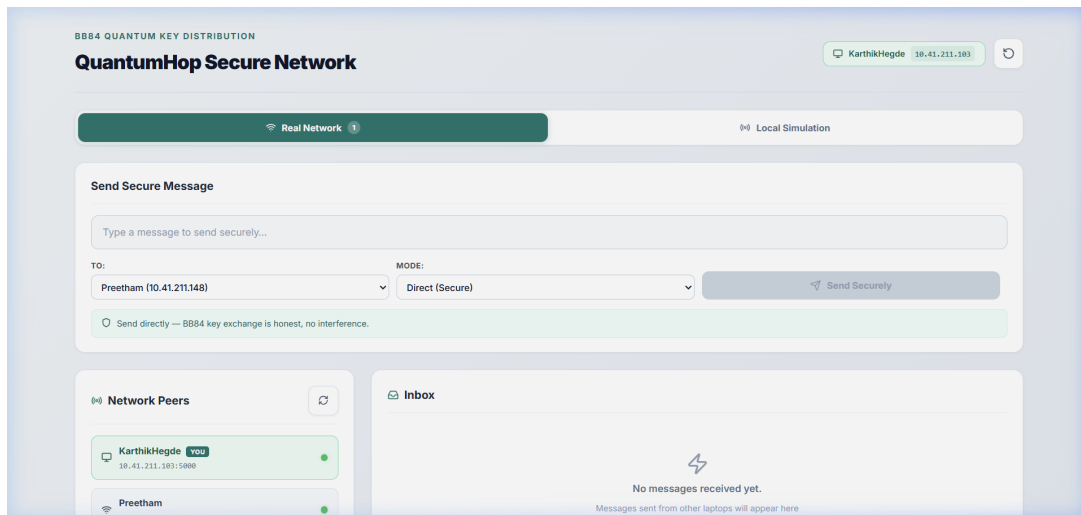


Figure 5.1: Real Network Dashboard View showing Peer Discovery and Messaging

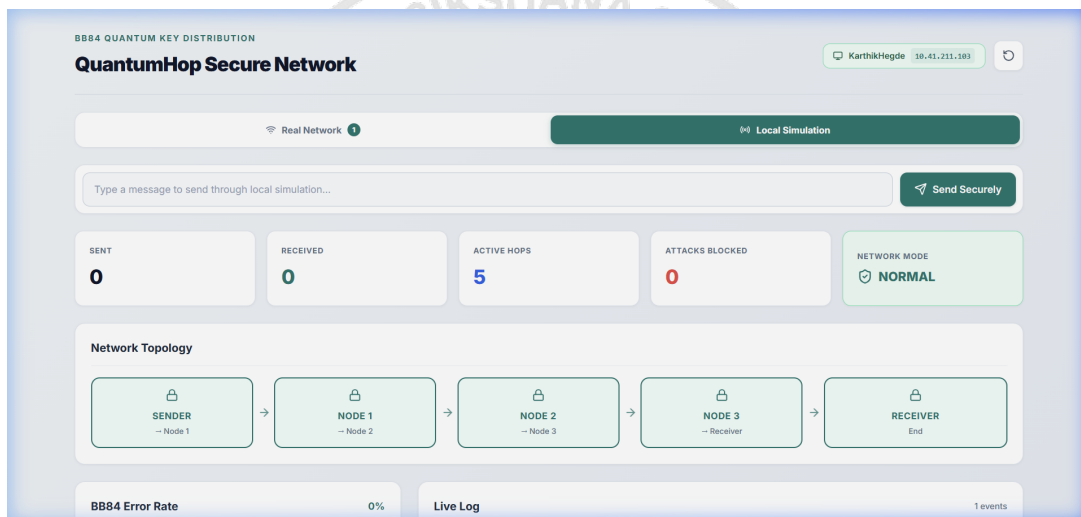


Figure 5.2: Local Simulation View showing Step-by-step BB84 Execution

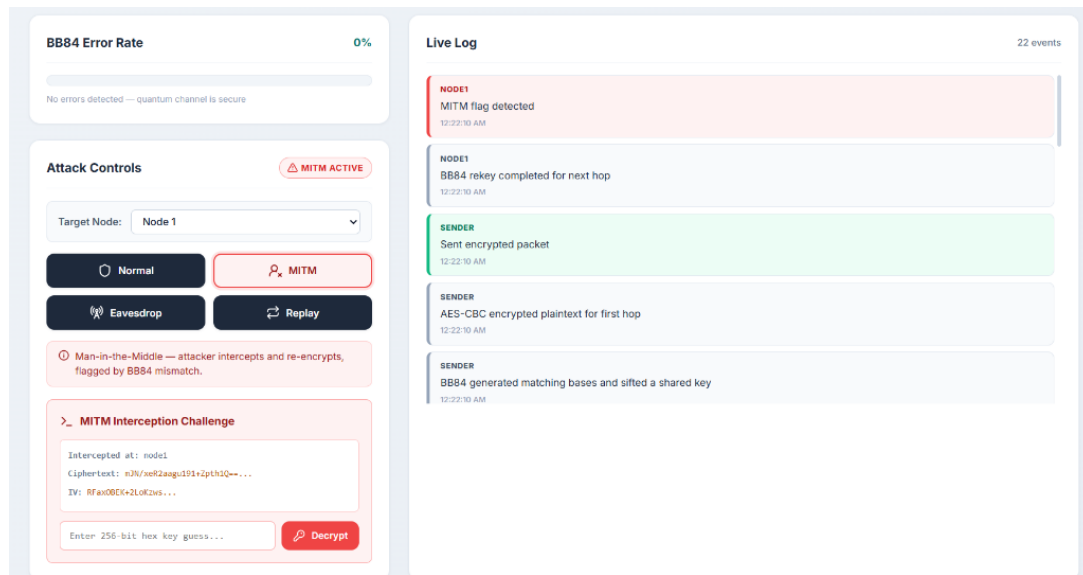


Figure 5.3: Interactive Man-in-the-Middle Attack Interception Challenge and Live Log View

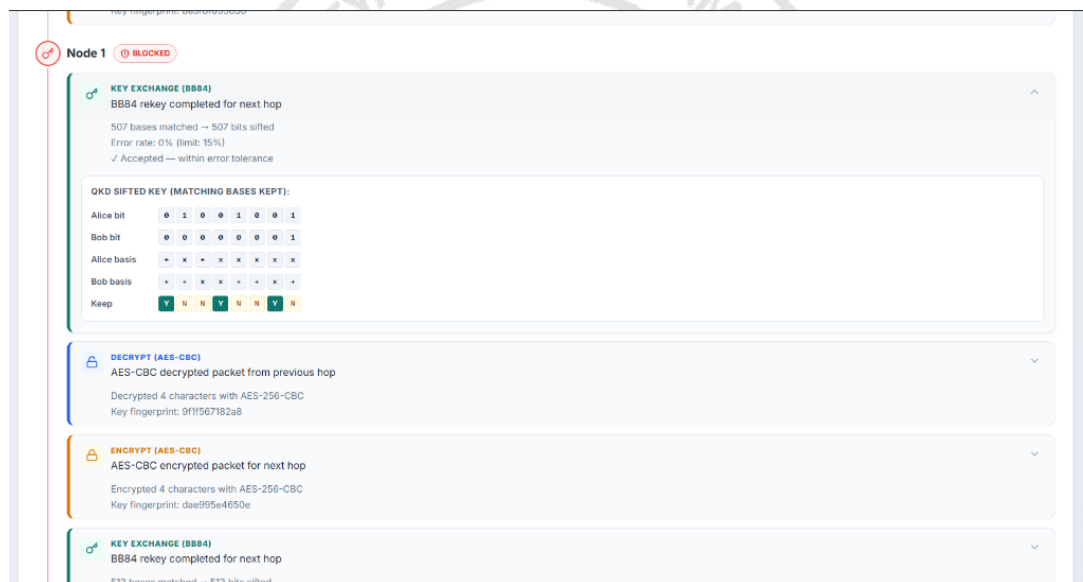


Figure 5.4: Simulation Hop Details showing QKD Sifted Key Table and AES Decryption/Encryption Blocks

5.2.8 Summary of Results

- BB84 key exchange correctly generates shared 256-bit AES keys and detects eavesdropping via error rate threshold.
- AES-256-CBC encryption successfully protects all message payloads transmitted over raw TCP sockets.
- Peer discovery via UDP broadcast correctly identifies all LAN machines within 3 sec-

onds.

- All four communication modes (direct, eavesdrop, MITM, replay) produce the expected cryptographic outcomes.
- The dashboard correctly displays all security events, inbox entries, and attack alerts in real time.
- The launcher script reliably kills stale processes and starts all services cleanly on every run.





Chapter 6

Conclusion

CHAPTER 6

CONCLUSION

The QuantumHop Secure Multi-Hop Communication System successfully demonstrates the integration of quantum-inspired key distribution with practical raw TCP socket programming. By simulating the BB84 Quantum Key Distribution protocol in software, the system derives fresh AES-256 encryption keys for every message without relying on classical Diffie-Hellman or RSA key agreement. The system operates on real physical laptops over a standard LAN and correctly detects three categories of network attacks: passive eavesdropping, Man-in-the-Middle relay, and replay attacks.

The project proves that the core physical principle of BB84 — that any eavesdropping attempt on the quantum channel introduces statistically detectable errors — can be translated into a practical software security mechanism. The configurable error rate threshold provides a tunable security margin, and the nonce-based replay protection adds a complementary layer against packet capture and retransmission attacks.

The interactive React dashboard provides a rich, real-time educational experience. Students and practitioners can observe every step of the BB84 protocol, watch AES keys being derived, see messages arriving in the inbox, and trigger attack scenarios at the click of a button. This makes the system uniquely suitable for laboratory demonstrations, project exhibitions, and self-paced learning of quantum cryptography concepts.

6.1 Limitations

- 1. Simulated Quantum Channel:** The BB84 protocol is simulated in classical software. A real QKD system uses single photons transmitted over a dedicated optical channel, providing information-theoretic security that no classical computation can break. The software simulation provides conceptual fidelity but not physical security.
- 2. Key Transmission in Packet:** In the current implementation, the sender includes the derived BB84 key in the JSON envelope so that the receiver can decrypt the message using the same key. In a true QKD system, Alice and Bob derive identical keys independently through quantum measurement; no key is ever transmitted. This design decision was necessary because two independent software BB84 runs would produce different keys.
- 3. In-Memory Storage:** The inbox, event log, and nonce registry are stored in memory

and are lost on restart. A persistent backend would require a database such as SQLite or Redis.

4. **Single-Threaded BB84 Simulation:** The BB84 simulation runs synchronously on the Flask request thread. For very long messages or high-volume systems, this could become a bottleneck.
5. **No TLS on Flask API:** The Flask API (port 5000) is served over plain HTTP. While inter-machine message traffic uses AES-encrypted socket payloads, the browser-to-Flask communication is not encrypted. In a production deployment, HTTPS would be required.
6. **Predefined Port Configuration:** All ports (5000, 5010, 5173, 5555) are hardcoded. Deployment in environments with strict firewall policies would require configuration changes.

6.2 Future Enhancements

1. **Independent Key Derivation:** Implement a protocol where Alice and Bob independently simulate BB84 with a shared random seed, deriving the same key without transmitting it. This would more faithfully model actual QKD without specialised hardware.
2. **Integration with Real QKD Hardware:** Connect the system to commercially available QKD hardware (such as ID Quantique or Toshiba QKD systems) to replace the simulation with actual photon-based key distribution.
3. **Persistent Storage:** Replace in-memory inbox and event store with SQLite or Redis to persist messages and logs across restarts.
4. **End-to-End TLS:** Wrap the Flask API with HTTPS using a self-signed certificate for secure browser-to-backend communication.
5. **Group Messaging:** Extend the socket layer to support broadcast or multicast delivery to multiple peers simultaneously using separate per-recipient BB84 key exchanges.
6. **Multi-Hop Routing:** Implement an onion-routing style multi-hop path where a message passes through several intermediate nodes, each encrypting with its own BB84-derived key, to demonstrate quantum-secured multi-hop communication.

7. **Mobile Application:** Port the dashboard to a React Native or Flutter mobile application so that smartphones can participate in the QuantumHop network.
8. **Advanced Attack Simulation:** Add more sophisticated attack scenarios such as photon number splitting, intercept-and-resend with partial basis knowledge, and denial-of-service on the discovery port.
9. **Audit Logging:** Maintain a persistent, tamper-evident log of all security events using an append-only data structure, enabling post-incident forensic analysis.
10. **Certificate-Based Peer Authentication:** Add a lightweight PKI so that peers present certificates during discovery, preventing a rogue device from impersonating a legitimate node.

In conclusion, the QuantumHop project successfully bridges the gap between quantum cryptography theory and practical network security implementation. It demonstrates that quantum-inspired key exchange can be integrated into standard socket programming environments to provide meaningful protection against eavesdropping, relay, and replay attacks. The system serves as both a functional prototype and a compelling educational platform for the next generation of information security practitioners.



Appendix A

Code Snippets

APPENDIX A

CODE SNIPPETS

A.1 BB84 Key Exchange — Core Function

The following code shows the `establish_key()` function that drives the quantum-inspired key distribution in the QuantumHop system.

```
def establish_key(
    requested_key_bits: int = config.BB84_KEY_BITS,
    threshold: float = config.ERROR_THRESHOLD,
    eavesdrop: bool = False,
    bit_flip_rate: float = 0.0,
) -> BB84Result:
    rounds = max(requested_key_bits * 4, config.BB84_SAMPLE_SIZE * 4)
    alice_bits = generate_bits(rounds)
    alice_bases = generate_bases(rounds)
    bob_bases = generate_bases(rounds)
    bob_bits = measure_bits(
        alice_bits, alice_bases, bob_bases,
        eavesdrop=eavesdrop, bit_flip_rate=bit_flip_rate
    )
    alice_sifted, bob_sifted = sift_key(
        alice_bits, bob_bits, alice_bases, bob_bases
    )
    compared_bits = min(config.BB84_SAMPLE_SIZE, len(alice_sifted))
    error_rate = calculate_error_rate(
        alice_sifted, bob_sifted, compared_bits
    )
    key_material = bob_sifted[compared_bits:] or bob_sifted
    return BB84Result(
        key=derive_key(key_material),
        error_rate=error_rate,
        accepted=error_rate <= threshold,
```

```
...  
)
```

A.2 Raw TCP Socket — Send Envelope

```
def _send_envelope(host: str, port: int, envelope: dict) -> dict:  
    with socket.create_connection(  
        (host, port), timeout=config.SOCKET_TIMEOUT_SECONDS  
    ) as sock:  
        data = json.dumps(envelope).encode("utf-8")  
        sock.sendall(data)  
        sock.shutdown(socket.SHUT_WR)  
        chunks = []  
        while True:  
            chunk = sock.recv(config.SOCKET_BUFFER_SIZE)  
            if not chunk:  
                break  
            chunks.append(chunk)  
        return json.loads(b"".join(chunks).decode("utf-8"))
```

A.3 Attack Detection — Handle Message

```
nonce_result = attack_detector.register_nonce(nonce)  
attack_detected = False  
attack_type = ""  
  
if not nonce_result.ok:  
    attack_detected = True  
    attack_type = "Replay Attack - duplicate nonce detected"  
elif is_mitm:  
    attack_detected = True  
    attack_type = "Man-in-the-Middle - packet relayed through attacker"  
elif attack_detector.is_error_rate_attack(error_rate):  
    attack_detected = True
```



```
attack_type = (  
    f"Eavesdropping - BB84 error rate "  
    f"{round(error_rate * 100)}% exceeded "  
    f"{round(config.ERROR_THRESHOLD * 100)}% threshold"  
)
```





Appendix B

Screenshots

APPENDIX B

SCREENSHOTS

B.1 Real Network Dashboard

Figure B.1 shows the QuantumHop dashboard in Real Network mode with a discovered peer and the inbox panel visible.

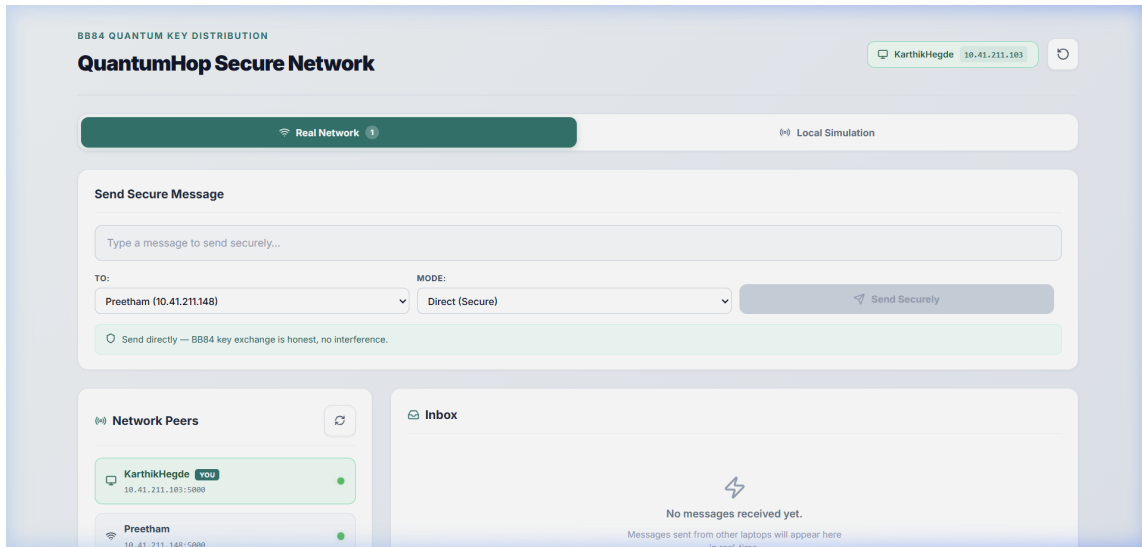


Figure B.1: QuantumHop Real Network Dashboard — Live peer discovery and inbox

B.2 Local Simulation Dashboard

Figure B.2 shows the dashboard in Local Simulation mode, displaying the step-by-step key exchange progress and the network topology diagram.

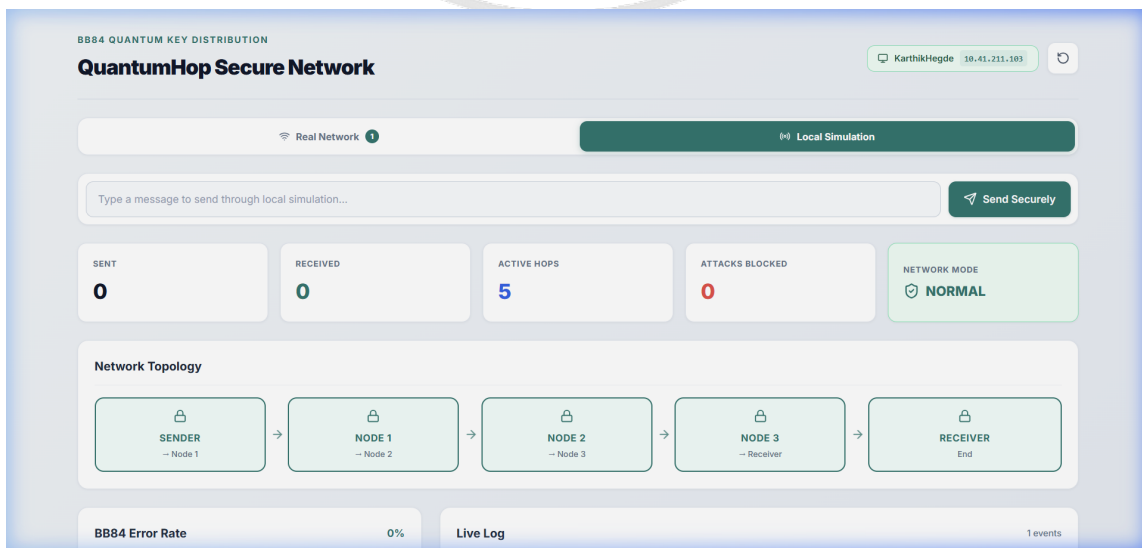


Figure B.2: QuantumHop Local Simulation Dashboard — Topology and QKD progress

B.3 Man-in-the-Middle Attack Interception

Figure B.3 shows the interactive MITM challenge interface where users can guess the intercepted 256-bit key, along with the live server security logs.

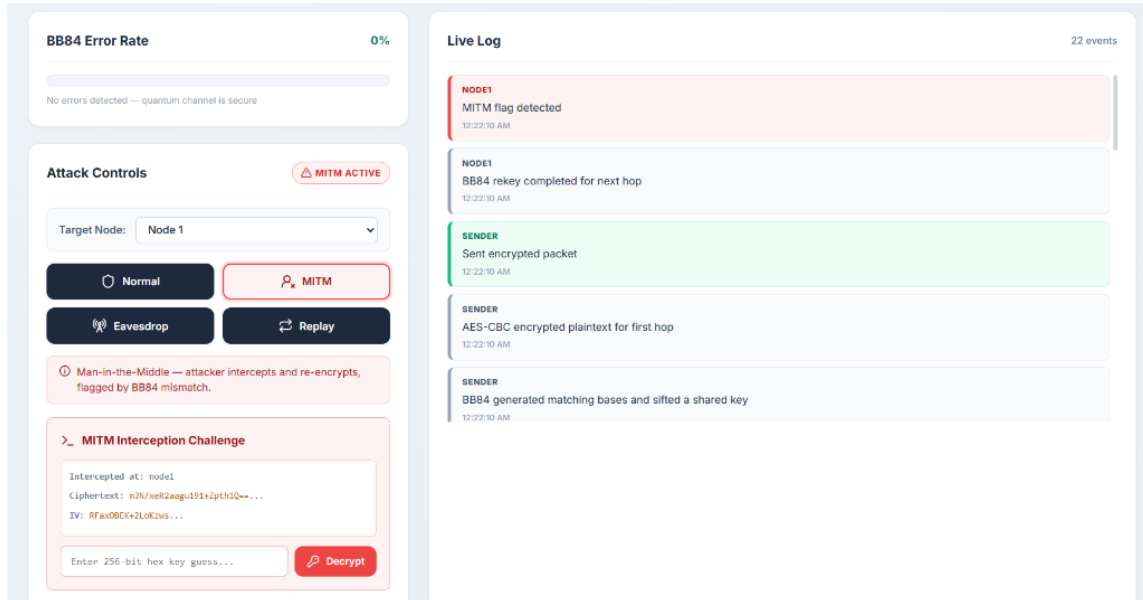


Figure B.3: Interactive MITM Challenge and Live Server Logs

B.4 Cryptographic and Rekey Hop Details

Figure B.4 shows the full hop-by-hop cryptographic breakdown, including the BB84 basis matching table, AES decryption, and AES encryption steps.

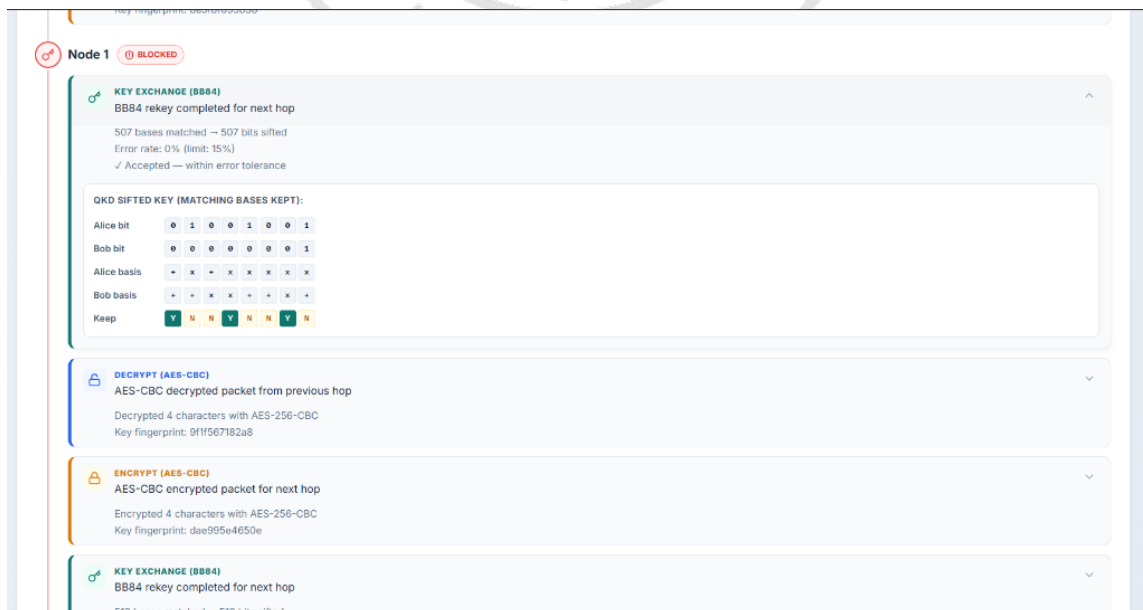


Figure B.4: Step-by-step QKD Basis Sifting and AES Hop Transformations

BIBLIOGRAPHY

- [1] C. H. Bennett and G. Brassard, “Quantum cryptography: Public key distribution and coin tossing,” *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, pp. 175–179, 1984.
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2010, ISBN: 978-1107002173.
- [3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th. Pearson Education, 2017, ISBN: 978-0134444284.
- [4] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, “Quantum cryptography,” *Reviews of Modern Physics*, vol. 74, pp. 145–195, 2002. DOI: 10.1103/RevModPhys.74.145
- [5] W. R. Stevens, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 2nd. Prentice Hall, 1998, ISBN: 978-0139498763.
- [6] Python Software Foundation. “Socket — low-level networking interface,” Accessed: Dec. 1, 2024. [Online]. Available: <https://docs.python.org/3/library/socket.html>
- [7] National Institute of Standards and Technology. “Post-quantum cryptography standardization,” Accessed: Dec. 1, 2024. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [8] H.-K. Lo and H. F. Chau, “Unconditional security of quantum key distribution over arbitrarily long distances,” *Science*, vol. 283, no. 5410, pp. 2050–2056, 1999. DOI: 10.1126/science.283.5410.2050